

React Render HTML

React 的目標是通過多種方式在網頁中呈現 HTML。

React 通過使用一個名為 ReactDOM.render() 渲染函數產出 HTML。

render 渲染函數

該 ReactDOM.render() 函數有兩個參數，1. HTML 文件 2. HTML 元素標籤。

該函數的目的是在指定的 HTML 元素內顯示指定的 HTML 文件。

例子

在 id 為 root 的元素中顯示一個段落：

```
ReactDOM.render(<p>Hello</p>, document.getElementById('root'));
```

結果顯示在<div id="root">元素中：

```
<body>
  <div id="root"></div>
</body>
```

範例:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello React Render!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    ReactDOM.render(<p>Hello World p Tag</p>,
document.getElementById('root'));
  </script>
</body>
</html>
```

範例:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello React Render!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>

<body>
  <div id="root"></div>
  <script type="text/babel">
    const myelement = (
      <table>
        <tr>
          <th>Name</th>
        </tr>
        <tr>
          <td>John</td>
        </tr>
        <tr>
          <td>Elsa</td>
        </tr>
      </table>
    );

    ReactDOM.render(myelement, document.getElementById('root'));
  </script>
</body>
</html>
```

React JSX

JSX 代表 JavaScript XML。

JSX 允許我們在 React 中編寫 HTML。

JSX 在 React 中編寫和添加 HTML 變得更加容易。

JSX 程式碼

JSX 允許我們在 JavaScript 中編寫 HTML 元素並將它們放置在 DOM 中，而無需任何 createElement() 和/或 appendChild()方法。

JSX 將 HTML 標籤轉換為 React 元素。

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8" />
  <title>Hello React JSX!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    const myElement = <h1>I Love JSX!</h1>;
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(myElement);
  </script>
</body>
</html>
```

Expression in JSX

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8" />
  <title>Hello React JSX!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>

<body>
  <div id="root"></div>

  <script type="text/babel">
    const myElement = <h1>React is {5 + 5} times better with JSX</h1>;
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(myElement);

  </script>
</body>
</html>
```

Class 屬性:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8" />
  <title>Hello React JSX!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
```

```

src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
  <style>
    .myclass {
      color: blue;
    }
  </style>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    const myElement = <h1 className="myclass">Hello World</h1>;
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(myElement);
  </script>
</body>
</html>

```

JSX 邏輯運算

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello React JSX!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
  <div id="root"></div>

  <script type="text/babel">

```

```
const x = Math.random() * 11;  
alert(x);  
const myElement = <h1>{(x) < 5 ? "Hello" : "Goodbye"}</h1>;  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(myElement);
```

```
</script>  
</body>  
</html>
```

React Components:

Component(組件)是獨立且可重用的程式碼。它們的作用與 JavaScript 函數相同，但獨立工作並返回 HTML。

組件有兩種類型，類別組件和函數組件

建立 React 組件時，組件名稱必須以大寫字母開頭。

Class (類別)組件

類別組件必須包含該 `extends React.Component` 語句。此語句建立對

`React.Component` 的繼承，並讓您的組件訪問 `React.Component` 的功能。

該組件還需要一個 `render()`方法，返回 HTML。

```
class Car extends React.Component {  
  render() {  
    return <h2>Hi, I am a Car!</h2>;  
  }  
}
```

Function(函數)組件

這是與上面相同的範例，但是使用 `Function` 組件建立。

`Function` 組件也返回 HTML，其行為方式與 `Class` 組件非常相似，但是 `Function` 組件可以使用更少的程式碼編寫，更容易理解。

例子

建立一個名為 `Car`

```
function Car() {  
  return <h2>Hi, I am a Car!</h2>;  
}
```

render(渲染)組件

現在 React 應用程式有一個名為 `Car` 的組件，它返回一個 `<h2>`元素。

您的應用程式中使用此組件，請使用與普通 HTML 類似的語法：`<Car />`

例子

在 root 元素中顯示 Car 組件：

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello React Components!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    function Car() {
      return <h2>Hi, I am a Car!</h2>;
    }
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<Car />);
  </script>
</body>
</html>
```

組件 props

組件可以作為傳遞 props(屬性)。

props 就像函數參數，將它們作為屬性發送到組件中。

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8" />
  <title>Hello React Components!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
```



```

    <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
  <div id="root"></div>

  <script type="text/babel">
    function Car(ps) {
      return <h2>I am a {ps.color} Car!</h2>;
    }
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<Car color="red" />);
  </script>
</body>

</html>

```

組件中置放組件:

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello React Components!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>
<body>
  <div id="root"></div>

  <script type="text/babel">

```

```

function Car() {
  return <h2>I am a Car!</h2>;
}

function Garage() {
  return (
    <div>
      <h1>Who lives in my Garage?</h1>
      <Car />
    </div>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Garage />);
</script>
</body>
</html>

```

組件置放於文件中

React 可以重用程式碼，建議將組件拆分成單獨的文件。
 因此建立一個具有.js 文件副檔名的新文件並將程式碼放入其中：
 請注意，文件名必須以大寫字符開頭。

Car.js

```

function Car() {
  return <h2>Hi, I am a Car in Car.js!</h2>;
}
export default Car;

```

Car.html

```

<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8" />
  <title>Hello React Components!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>

```

```
<script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
<script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
  <div id="root"></div>
  <script type="text/babel" src="Car.js"></script>
  <script type="text/babel">
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<Car />);
  </script>
</body>
</html>
```

class(類別)組件

建立 **React** 組件時，組件名稱必須以大寫字母開頭。

該組件必須包含 `extends React.Component` 語句，該語句建立對 `React.Component` 的繼承，並使您的組件可以訪問 `React.Component` 的功能。

組件還需要一個 `render()` 方法，該方法返回 **HTML**。

例子

建立一個名為 `Car` 類別

```
class Car extends React.Component {
  render() {
    return <h2>Hi, I am a Car!</h2>;
  }
}
```

組件建構函式

如果你的組件中有一個 `constructor()` 函數，這個函數將在組件啟動時被使用。

建構函式是您啟動組件屬性的地方。

在 `React` 中，組件屬性應該保存在一個名為 `state` 內。

建構函式也是您通過包含 `super()` 執行父組件建構函式的語句繼承的地方，您的組件可以訪問父組件的所有函數（`React.Component`）。

例子

在 `Car` 組件中建一個建構函式，並添加一個顏色屬性：

在 `render()` 函數中使用顏色屬性：

例子

```
class Car extends React.Component {
  constructor() {
    super();
    this.state = {color: "red"};
  }
  render() {
    return <h2>I am a {this.state.color} Car!</h2>;
  }
}
```

範例:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello React Components!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
```

```

</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    class Car extends React.Component {
      constructor() {
        super();
        this.state = { color: "Blue" };
      }
      render() {
        return <h2>I am a {this.state.color} Car in class
Component!</h2>;
      }
    }
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<Car />);
  </script>
</body>
</html>

```

建構函式中的 **props**

如果你的組件有一個建構函式，**props** 應該必須傳遞給父建構函式，並通過 **super()** 方法傳遞給 **React.Component**。

例子

```

class Car extends React.Component {
  constructor(props) {
    super(props);
  }
  render() {
    return <h2>I am a {this.props.model}</h2>;
  }
}

```

範例:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8" />
  <title>Hello React Components!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    class Car extends React.Component {
      constructor(props) {
        super(props);
      }
      render() {
        return <h2>I am a {this.props.model} in Class props!</h2>;
      }
    }
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<Car model=" Demo Car " />);
  </script>
</body>
</html>
```

React Class Component State:

React 類別組件有一個內置 **state** 物件。

您可能已經注意到我們 **state** 之前在組件建構函數部分中使用過。

該 **state** 物件是您存儲屬於組件的屬性值的地方。

當 **state** 物件改變時，組件會重新渲染。

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello React Components!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
</head>
<body>
  <div id="root"></div>

  <script type="text/babel">
    class Car extends React.Component {
      constructor(props) {
        super(props);
        this.state = {
          brand: "Ford",
          model: "Steel",
          color: "Black",
          year: 2023
        };
      }
      render() {
        return (
          <div>
            <h1>My {this.state.brand}</h1>
            <p>
              It is a {this.state.color}
              {this.state.model}
              from {this.state.year}.
            </p>
          </div>
        );
      }
    }
  </script>

```

```

    }

    ReactDOM.render(<Car />, document.getElementById('root'));
  </script>
</body>
</html>

```

改變 state 物件

要更改狀態物件中的值，請使用 `this.setState()` 方法。

當物件中的值 `state` 更改時，組件將重新渲染，這意味著輸出將根據新值更改。

範例加入 **按鈕觸發事件** 函式：

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello React Components!</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>
<body>
  <div id="root"></div>

  <script type="text/babel">

```

```

    class Car extends React.Component {
      constructor(props) {
        super(props);
        this.state = {
          brand: "Ford",
          model: "Mustang",
          color: "red",
          year: 2023

```



```

        };
    }
    changeColor = () => {
        this.setState({ color: "Blue" });
    }
    render() {
        return (
            <div>
                <h1>My {this.state.brand}</h1>
                <p>
                    It is a {this.state.color}
                    {this.state.model}
                    from {this.state.year}.
                </p>
                <button
                    type="button"
                    onClick={this.changeColor}
                >Change color</button>
            </div>
        );
    }
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car />);
</script>
</body>
</html>

```

建一個名為 `carName` 的變數並將其發送到 `Car` 組件 `props` 內：

```

<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8" />
    <title>Hello React Components!</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script

```

```
src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
  <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>

</head>

<body>
  <div id="root"></div>

  <script type="text/babel">
    function Car(props) {
      return <h2>I am a {props.brand}!</h2>;
    }

    function Garage() {
      const carName = "Ford";
      return (
        <div>
          <h1>Who lives in my garage?</h1>
          <Car brand={carName} />
        </div>
      );
    }

    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<Garage />);
  </script>
</body>
</html>
```